03 DFS

Lab 2b: Depth-First Search

Same lab, same query: A to H. Now change one line of your BFS program to make is depth-first.

Answer this: DFS visits fewer nodes here than BFS. Is DFS therefore the better algorithm? Move the goal from under E to under G2 and re-run before answer.

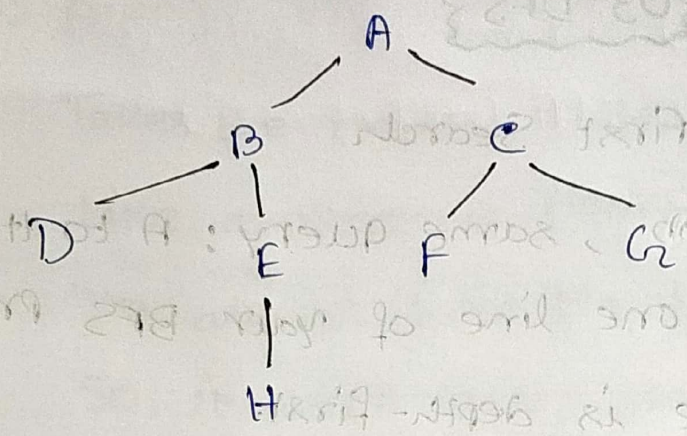
=> • What is DFS? :—

DFS stands for Depth-First-Search

"It will go as deep as possible in one path first. If it find a dead end, it will come back and try another path."

So DFS :— i) chooses one path
ii) Goes deeper and deeper
iii) It stuck, comes back (Backtracking).
iv) Tries another path

• Tree :—



we want to go from A → H.

- Step-by-Step DFS :-

i) Initial Stack ⇒ [A]

ii) Step 1 ⇒ POP A

visited : A

Push C then B (so B is visited first)

Stack : [C, B]

iii) Step 2 ⇒ POP B

visited : A, B

Push E then D

Stack : [C, E, D]

iv) Step 3 ⇒ POP D

visited : A, B, D

D has no children

Stack : [C, E]

v) Step 4 ⇒ POP E

visited : A, B, D, E

Push H

stack : [C, H]

vi) Step 5 ⇒ POP H

visited : A, B, D, E, H

Goal Found !

Path : A → B → E → H

order : A → B → D → E → H

Python Program :

```
1. graph = {
2.     'A' : ['B', 'C'],
3.     'B' : ['D', 'E'],
4.     'C' : ['F', 'G'],
5.     'D' : [],
6.     'E' : ['H'],
7.     'F' : [],
8.     'G' : [],
9.     'H' : []
10. }

11. stack = [('A', ['A'])]

12. while stack:
13.     node, path = stack.pop()
14.     if node == 'H':
15.         print("Path:", path)
16.         break
```

17. for child in reversed(graph[node]):
18. stack.append((child, path + [child]))

Output Path: ['A', 'B', 'E', 'H']

- Code Explanation :—

1. i) Creates a graph (tree).
   ii) Think of it as a map showing which node is connected to which node
2. A is connected to B and C
3. B " " " D " E
4. C " " " F " G
5. i) D has no children.
   ii) It is a dead end.
6. E is connected to H
7. 8. 9. F, G, H have no children.
10. Graph creation finished.
11. i) create a stack.
    ii) Put A inside it.
    iii) current Stack: [('A', ['A'])]
    iv) meaning: a) current node = A
    b) current Path = A
12. i) Run the loop untill the stack becomes empty.

ii) If stack has something → continue.

iii) If stack is empty → stop.

13. i) Take out the top item from the stack.

ii) DFS always removes the last inserted item.

Eg Before: [[A, [A]]]

After pop: node = A

path = [A]

Stack become empty.

14. Check: "Did we reach H?"

15. If H is found, print the path.

Eg Path: ['A', 'B', 'E', 'H']

16. i) Stop the program

ii) Goal found.

17. Look at all children of the current node.

Eg For A: graph ['A']

gives [B, C]

reversed makes it [C, B]

why? i) Because stack works backwards

ii) This helps DFS visit B before C

18. i) Put the child into the stack

ii) Also store the new path.

Eg Current: node = A

path = [A]

Child = B

new item: (B, [A, B])

Child = C

new item: (C, [A, C])

Both are added to the stack.

Summary:— i) DFS uses a stack

ii) It goes deep into one path first.

- How BFS was Different? :—

BFS explores level by level.

visited: A
B
C
D
E
F
G
H

Total: 8 nodes

- Nodes visited By DFS: —

Order: A
B
D
E
H

Total visited: 5 nodes

- Why Did DFS visit Fewer Nodes? :—

  i) Because H is under E

  ii) DFS quickly went : A → B → E → H

  and found the goal.

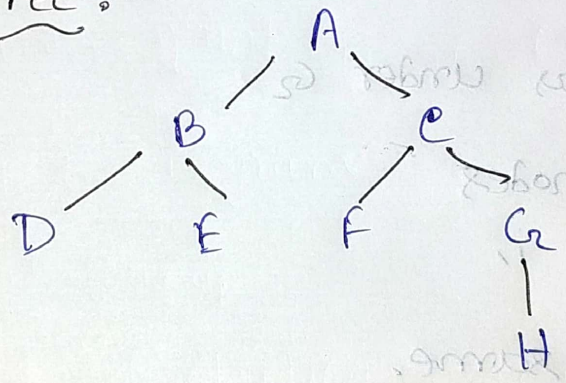
  iii) It did not need to visit : C
  F
  G2

- Does This Mean DFS IS Always Better? :—

  - NO
  - This example is only one case

  Let's move H under G2

- New Tree :—



- DFS Again :— Traversal :
  A
  B
  C
  D
  E
  F
  G2
  H

  visited :— 8 nodes

- DFS Again :— Traversal : A
  B
  D
  E
  C
  F
  G
  H

Visited :— 8 nodes

- Observation :—

i) When H was under E

DFS = 5 nodes
BFS = 8 "

DFS looked better

ii) When H was under G

DFS = 8 nodes
BFS = 8 "

Both are same.

iii) The performance of DFS depends on where the goal node is located.

iv) Sometimes DFS is faster,
" BFS " better

v) Therefore DFS is not always the better algorithm.